COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S2
DATE PERFORMED:	09/18/26	**DATE SUBMITTED:**	09/23/26
NAME:	Prince Benjamin S. Villaruel	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 9.1 Tree ADT

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 9-1: General Tree

Code Snippet	GeneralTree.h > isLeaf()
	#ifndef GENERALTREE_H
	#define GENERALTREE_H
	
	#include<iostream>
	template <typename T>
	
	class GeneralTree{
	private:
	T key;
	GeneralTree<T>* firstChild;
	GeneralTree<T>* nextSibling;
	
	public:
	//constructor
	GeneralTree(T rootObj);
	
	//inserting a child node, gets added after the existing ones and returns the new node
	GeneralTree<T>* insertChild(T newNode);
	
	GeneralTree<T>* getFirstChild();
	GeneralTree<T>* getNextSibling();
	
	T getRootVal();
	void setRootVal(T obj);
	
	bool isLeaf();
	
	//just to check the tree looks right
	void printTree(int depth = 0);
	};
	
	template <typename T>
	GeneralTree<T>::GeneralTree(T rootObj) {
	key = rootObj;
	firstChild = nullptr;
	nextSibling = nullptr;
	}
	template <typename T>
	GeneralTree<T>* GeneralTree<T>::insertChild(T newNode) {
	GeneralTree<T>* newChild = new GeneralTree<T>(newNode);
	
	if (firstChild == nullptr) {
	//no kids yet so this is the first one
	firstChild = newChild;
	} else {
	//otherwise go to the end of the sibling list and stick it there
	GeneralTree<T>* temp = firstChild;
	while (temp->nextSibling != nullptr) {
	temp = temp->nextSibling;
	}
	temp->nextSibling = newChild;
	}
	
	return newChild;
	}

```cpp
template <typename T>
GeneralTree<T>* GeneralTree<T>::getFirstChild(){
    return firstChild;
}

template <typename T>
GeneralTree<T>* GeneralTree<T>::getNextSibling(){
    return nextSibling;
}

template <typename T>
T GeneralTree<T>::getRootVal() {
    return key;
}

template <typename T>
void GeneralTree<T>::setRootVal(T obj) {
    key = obj;
}

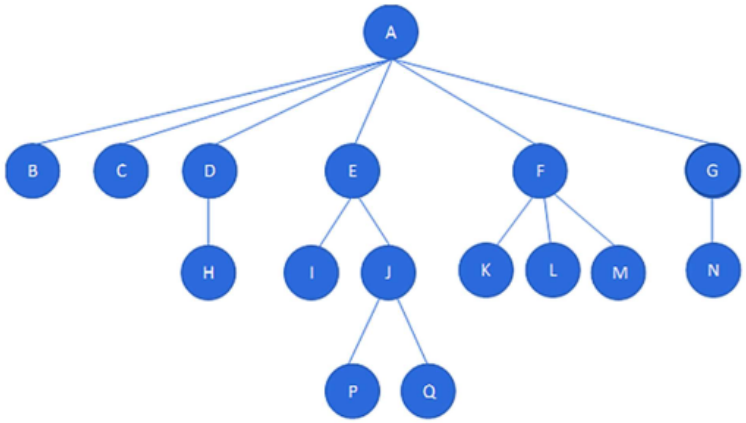
template <typename T>
bool GeneralTree<T>::isLeaf() {
    return firstChild == nullptr;
}

template <typename T>
void GeneralTree<T>::printTree(int depth) {
    for (int i = 0; i < depth; i++) {
        std::cout << "  ";
    }
    std::cout << key << std::endl;

    if (firstChild != nullptr) {
        firstChild->printTree(depth + 1); //go down
    }
    if (nextSibling != nullptr) {
        nextSibling->printTree(depth); //next one over
    }
}

#endif
```

Tree Diagram



Observation

I created a C++ node defining a template GeneralTree class using a first-child/next-sibling representation.

Table 9-2

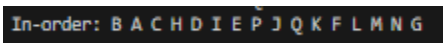
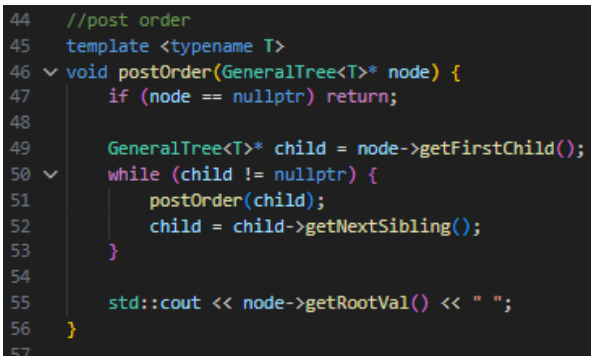
Node	Height	Depth

A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
O	0	3
P	0	3
Q	0	3

Table 9-3

Pre-order	ABCDHEIJPQFKLMGN
In-order	BACHDIEPJQKFLMNG
Post-order	BCHDIPQJEKLMFNGA

Table 9-4

Pre-order Snippet	
Pre-order Output	
In-order Snippet	
In-order Output	
Post-order Snippet	
Post-order Output	
Observation	For this task, I noticed that the code I implemented provides the same results in regards to table 9-3. In this approach, each node stores a key value alongside firstChild and nextSibling pointers.

```
Pre-order: A B C D H E I J P Q F K L M G N
```

```
In-order: B A C H D I E P J Q K F L M N G
```

```
Post-order: B C H D I P Q J E K L M F N G A
```

Table 9-5

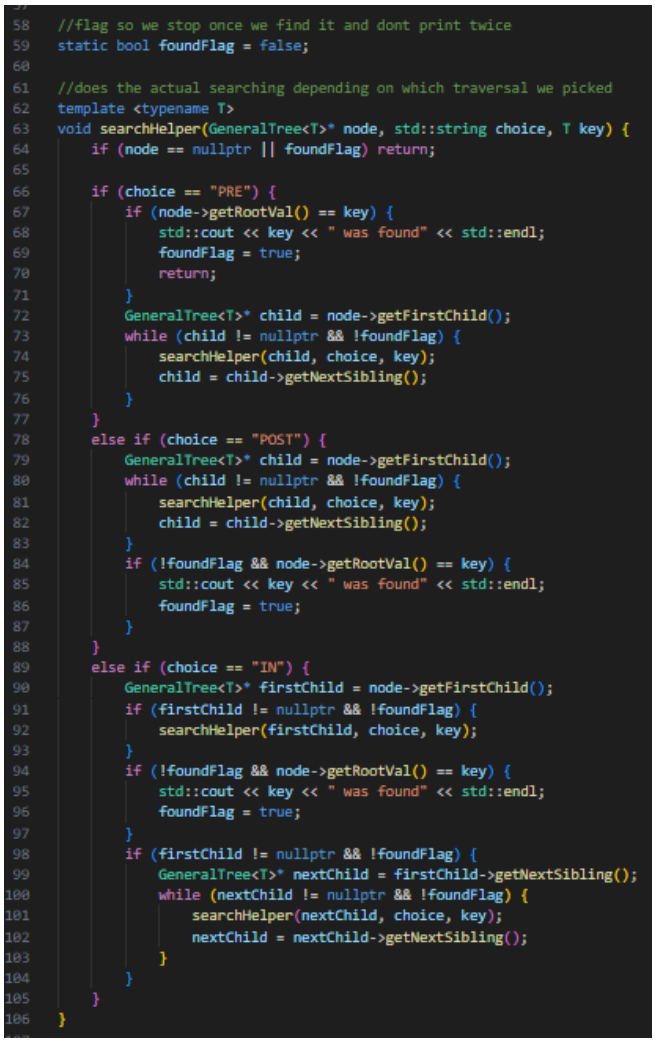
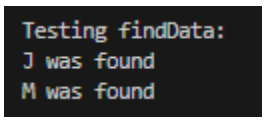
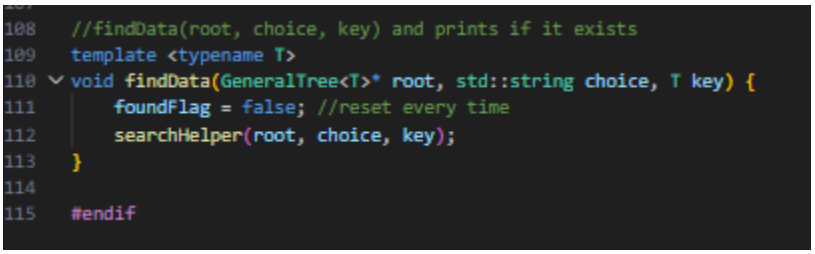
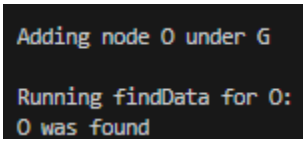
Code Snippet	
Output	
Observations	For this task, I created a helper function named searchHelper that searches for a specified key in a general tree using a chosen traversal order ("PRE", "POST", or "IN"). It uses a static boolean flag foundFlag to early-exit recursion once the target is located.

Table 9-6

Code Snippet	
Output	
Observations	For this task, I created a wrapper function named findData wherein it initiates a search operation within the general tree. It accepts the root node pointer, a traversal choice string, and the target key.

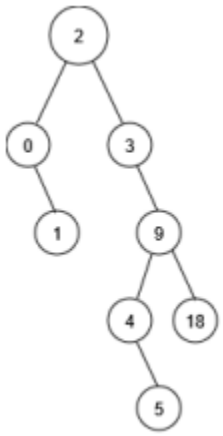
References:

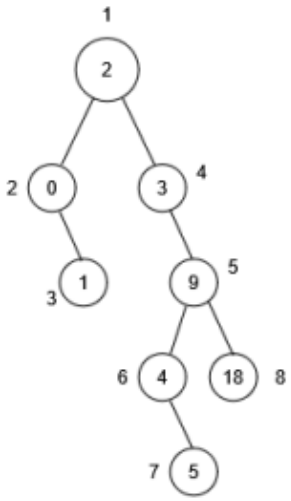
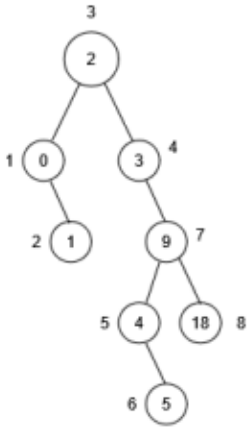
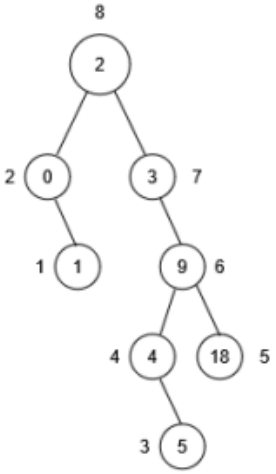
https://www.geeksforgeeks.org/dsa/difference-between-general-tree-and-binary-tree/

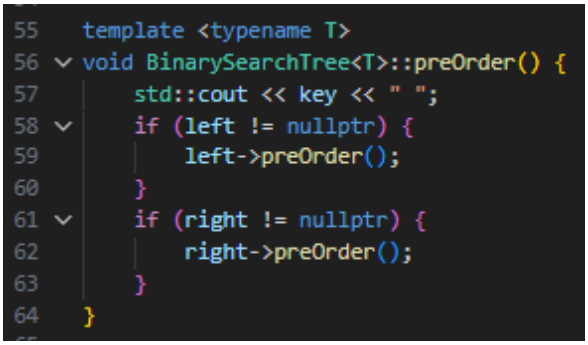
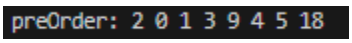
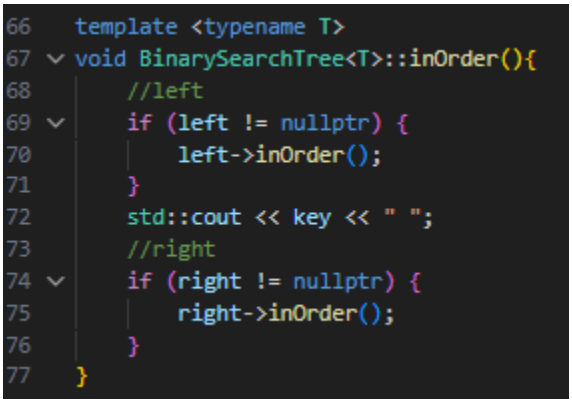
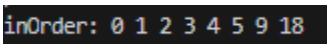
https://studyglance.in/ds/display.php?tno=22&topic=Introduction-to-Tree

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Traversal	Diagram
Tree Diagram	

Pre-order	
In-order	
Post-order	

Code Snippet	
Output	
Code Snippet	
Output	
Code Snippet	
Output	
Observations	For the supplementary activity, it is a simple implementation of BST that shows depth-first

	traversals using direct left and right pointers. In addition, I also noticed that pre-order processes the root before subtrees, in-order visits the left subtree, root, and right subtree to yield a sorted order, and post-order processes subtrees before the root.

Conclusion

Overall, HOA 9.1 offered great practical insight into C++ tree data structures, even though balancing the conceptual differences between GTs, BTs, and BSTs was definitely hard and confusing at first. The BST was the structure I understood the most, as its two-pointer setup makes traversal and sorting intuitive compared to the other tree structures, especially GTs. Tracing pre-order, in-order, and post-order outputs across both GT and BST, along with implementing conditional node search helpers, properly clarified how fundamental traversal logic adapts across different tree architectures.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.